

AI-Powered Edge-Based Road Accident Detection and Road Accident Risk Index (RAR) System with Integrated Urban Microclimate Monitoring for Sri Lanka

Abheek Pathirana
Springfield College Nugegoda
Sri Lanka
Email: abheek.pathirana@gmail.com

Abstract—Road traffic incidents remain a critical public safety concern in Sri Lanka [1], further complicated by delayed emergency notification and limited bystander intervention due to legal ambiguities. This paper presents a low-cost, edge-deployable artificial intelligence system for real-time accident detection and proactive risk estimation using a novel Road Accident Risk Index (RAR). The system integrates a fine-tuned YOLOv5n [2] accident detection model (mAP@50: 0.8027; mAP@95: 0.5170) and a YOLOv8n [3]-based vehicle detection and tracking module deployed in ONNX format on a Raspberry Pi 4. In addition, the platform incorporates NB-IoT communication and environmental sensing for street-level Urban Heat Island (UHI) [4] monitoring. The system supports Sustainable Development Goals (SDG) 3.6 and 13 by enabling automated emergency alerts and scalable microclimate data collection. Architectural decisions, model selection trade-offs, and alternative approaches are analyzed. Experimental validation demonstrates feasibility on resource-constrained hardware.

Index Terms—Road Accident Risk Index (RAR), Accident Detection, Edge AI, YOLOv5n, YOLOv8n, Urban Heat Island, Embedded Vision, NB-IoT, Smart Infrastructure, SDG 3.6, SDG 13

I. INTRODUCTION

Road traffic accidents represent a growing public health crisis in Sri Lanka, with thousands of fatalities and injuries recorded annually. According to the World Health Organization (WHO), road traffic crashes were responsible for over 2,500 deaths in 2025 alone [5], with a disproportionately high number occurring in urban centers such as Colombo and Kandy [1]. Beyond fatalities, accidents impose economic costs through lost productivity, emergency medical care, property damage, and long-term disability care. Critically, first-responder response times in Sri Lanka average between 20–40 minutes, depending on urban density and traffic conditions, significantly increasing the likelihood of fatal outcomes in severe incidents [6]. Studies show that for severe trauma cases, each minute delay in receiving emergency care reduces survival probability by up to 1-2%, meaning even small reductions in response time can save multiple lives per year [7].

Multiple systemic challenges contribute to these delays. Ambulance availability is limited in both urban and suburban areas, often exacerbated by congestion during peak hours. Bystander assistance is inconsistent due to legal ambiguities in the Sri Lankan Penal Code that may implicate helpers in court proceedings. Cultural hesitation, insufficient first-aid knowledge, and poor traffic discipline further hinder immediate accident response. Consequently, real-time automated detection systems have the potential to dramatically reduce the time between incident occurrence and notification of emergency services, thereby improving survival rates.

Beyond the immediate safety concerns, urban infrastructure in Sri Lanka faces additional challenges. High-density neighborhoods experience the Urban Heat Island (UHI) effect, with localized temperatures rising several degrees above surrounding rural areas due to heat retention by asphalt, concrete structures, and vehicular emissions [4]. Elevated temperatures can exacerbate accident risk by contributing to driver fatigue, impaired concentration, and increased road surface degradation. However, current meteorological monitoring infrastructure provides only coarse-grained data at city or district level, failing to capture street-level microclimates that may correlate with accident hotspots. Integrating microclimate data collection into road accident monitoring systems offers both environmental research insights and practical safety applications.

A. Personal Motivation

The motivation for this work arises from direct observation of Sri Lankan traffic conditions and the desire to leverage low-cost AI for societal benefit. Witnessing prolonged delays in emergency response, encountering accident-prone intersections, and observing the impact of urban microclimates inspired a dual-focus solution: real-time accident detection and street-level environmental monitoring. This work combines local relevance with an educational motivation to demonstrate practical AI deployment on constrained hardware, bridging the gap between academic research and societal impact.

B. Research Objectives

This study aims to:

- 1) Develop a real-time edge-based accident detection system capable of accurate detection on low-resource hardware.
- 2) Propose and implement a Dynamic Road Accident Risk Index (RAR) for proactive risk assessment.
- 3) Integrate NB-IoT for automated emergency notification with human verification to reduce false positives.
- 4) Continuously monitor street-level microclimate conditions to enhance UHI analysis.
- 5) Evaluate system feasibility, computational efficiency, and practical deployment considerations in realistic urban scenarios.

C. Contribution Summary

The contributions of this paper include:

- A lightweight edge AI framework combining YOLOv5n accident detection and YOLOv8n vehicle tracking optimized for Raspberry Pi 4.
- The design and implementation of a Road Accident Risk Index (RAR) to estimate accident severity and potential outcomes.
- Integration of NB-IoT-based emergency alerts with GPS and environmental context.
- Street-level microclimate monitoring for urban heat and accident correlation studies.
- Evaluation of system feasibility on constrained hardware, with comprehensive training, deployment, and operational analysis.

II. RELATED WORK

A. Accident Detection Systems

Previous research on automated accident detection has predominantly focused on high-resource environments. For example, [8] demonstrated video-based detection using convolutional neural networks on high-performance GPUs, achieving high accuracy but at costs and complexity unsuitable for deployment in developing regions. Similarly, [9] presented edge-based systems for vehicle tracking, though without integration of environmental monitoring or proactive risk estimation. In the context of Sri Lanka, very few studies address the unique challenges of constrained infrastructure, variable network coverage, and traffic-induced delays in emergency response.

B. Edge AI in Resource-Constrained Environments

Edge AI allows model inference near the data source, reducing latency and network dependency. YOLO-family networks (v5, v8) have become prominent for real-time detection due to their lightweight architectures and high mAP scores. Prior studies [10] benchmarked YOLOv5 on embedded devices, showing feasibility for traffic monitoring with optimizations such as ONNX conversion and INT8 quantization. However, these studies rarely consider the full pipeline including risk scoring, alert transmission, and microclimate sensing, which

are critical for holistic accident management in urban Sri Lanka.

C. Urban Microclimate Monitoring and Accident Correlation

Recent studies suggest a correlation between microclimate variables and traffic incidents. High ambient temperatures and wet road surfaces increase accident probability, while localized wind patterns and lighting conditions affect driver behavior. Santamouris et al. [4] discuss methodologies for UHI mapping, but these approaches typically rely on sparse weather stations or satellite imagery. The integration of low-cost sensors for street-level UHI monitoring remains largely unexplored in conjunction with accident detection systems.

D. Limitations of Existing Solutions

While existing solutions provide valuable insights, several gaps persist:

- Dependence on high-performance GPUs unsuitable for large-scale, low-cost deployment.
- Absence of proactive risk assessment (no RAR or predictive scoring).
- Lack of integration with IoT communication protocols for real-time alerting.
- Minimal incorporation of microclimate data to inform environmental risk factors.

III. SYSTEM DESIGN

The proposed system is organized into four core layers: Vision-Based Detection, Risk Modeling (RAR), Communication, and Environmental Monitoring. Each layer addresses a specific operational requirement.

A. Vision-Based Detection Layer

The vision layer performs real-time object detection and accident identification on board a Raspberry pi 4. YOLOv5n is used for accident detection due to its balance between model size and mAP performance. YOLOv8n handles vehicle detection and tracking, maintaining unique IDs for dynamic RAR computation. Frame pre-processing (resizing, normalization, low-light correction) ensures robustness across various environmental conditions.

B. Risk Modeling Layer (RAR)

The Road Accident Risk Index computes a continuous risk score based on:

- 1) Distance and relative velocity estimates from monocular vision.
- 2) Bounding box height and width to approximate vehicle dimensions.
- 3) Time-to-Collision (TTC) estimation.
- 4) Optional integration of weather and environmental factors (rain, temperature, humidity).

The RAR score ranges from 0 (safe) to 10 (critical), facilitating proactive alerting before collisions occur.

C. RAR Computation

Distance and velocity estimation use monocular approximation:

$$D = \frac{F}{h}, \quad (1)$$

$$V = \frac{D_{prev} - D_{curr}}{\Delta t}, \quad (2)$$

$$TTC = \frac{D}{V}, \quad (3)$$

$$RAR = 10 \left(1 - \frac{TTC}{TTC_{max}} \right) \quad (4)$$

D. Communication Layer

NB-IoT is used for low-bandwidth, wide-area emergency notification. Data packets include GPS coordinates, timestamp, RAR score, and environmental metrics. Human verification ensures alerts are actionable and false positives minimized. Retry mechanisms handle transmission failures due to network variability.

E. Environmental Monitoring Layer

Microclimate sensors capture street-level temperature, humidity, and pressure. Geo-tagging allows correlation with accident incidence, identifying hotspots exacerbated by urban heat or poor ventilation. Longitudinal data supports both operational safety and urban planning initiatives.

F. Hardware Considerations

The system is designed for Raspberry Pi 4 (4GB RAM) [11] with an RTSP/IP camera or Pi camera module, SIM7000C [12] NB-IoT module, and SparkFun Weather Shield [13]. Solar support is included for stationary deployments. Hardware trade-offs emphasize low cost, low power consumption, and sufficient computational capability for real-time inference at 7 FPS before multithreading.

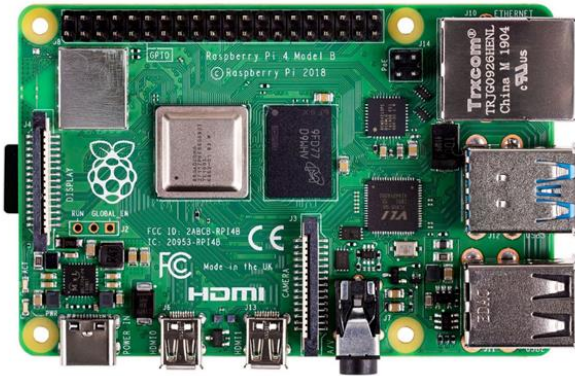


Fig. 1: Raspberry pi4.

IV. SYSTEM IMPLEMENTATION

A. Software Stack

Python 3.11 is used with OpenCV [14] for image capture and preprocessing, PyTorch for training, and ONNX Runtime

[15] for edge inference. NB-IoT communication is handled through AT commands on the SIM7000C. The RAR computation runs on Raspberry Pi 4 in real time, alternating YOLOv5n [2] and YOLOv8n [3] processing per frame to optimize FPS.

B. Data Preprocessing

Video frames are resized to 320×320 , normalized, and augmented (flip, brightness/contrast adjustment, Gaussian noise). GPS coordinates and timestamps are synchronized with video frames to maintain temporal coherence for RAR computation.

C. Model Deployment

YOLOv5n and YOLOv8n are exported to ONNX, enabling optimized inference on CPU. Vehicle tracking uses YOLOv8n's built-in tracker [16] to maintain object IDs. Edge deployment prioritizes minimal latency while maintaining above 80% mAP accuracy for accident detection.

V. METHODOLOGY

A. Selection of the Vision Models

After careful consideration of mAP scores, inference speeds, and memory requirements, **YOLOv5n** was selected as the base model for accident detection. Although **YOLOv8n** demonstrates a higher baseline mAP₅₀₋₉₅ of approximately 37.3% compared to YOLOv5n's $\sim 28.0\%$, deployment constraints on heavily resource-limited hardware necessitate prioritizing efficiency. YOLOv5n offers a lighter footprint, requiring only ~ 7.7 GFLOPs compared to YOLOv8n's ~ 8.7 GFLOPs, representing an 11.5% reduction in computational cost.

For the **RAR index**, YOLOv8n was chosen due to its substantial improvement in detection accuracy, providing a 33.2% higher mAP than the YOLOv5n base model and the built in tracking features that comes with yolov8. The 12% increase in computational demand is acceptable for this use case, as this model will not undergo additional fine-tuning.

B. Creation of the Datasets

For the development of the accident detection model, two distinct datasets were created to support the two-phase training strategy. The first, a general dataset, consisted of 10,000 images of traffic accidents collected from publicly available online sources. These images were annotated to include relevant bounding boxes only for vehicular based accidents.

The second, task-specific dataset was curated to represent challenging edge cases and scenarios commonly observed in Sri Lanka. This dataset comprised 4,000 handpicked images that captured diverse lighting conditions, weather variations, and unusual accident scenes. Both datasets were standardized to a uniform resolution and annotation format to ensure compatibility with the YOLOv5n model and to facilitate efficient training and evaluation.

C. Training of the Accident Detection Model

Our training methodology follows the established transfer learning paradigm, in which a model is first trained on a large, general dataset and then fine-tuned on a smaller, task-specific dataset to improve performance for the target application [17]–[19]. This two-phase approach has been shown to significantly enhance accuracy and sample efficiency across computer vision tasks such as image classification and object detection.

In our work, the YOLOv5n base model was initially trained on 10,000 generic annotated accident images collected from various online datasets for 120 epochs on an Apple Silicon M3 SoC. After this phase, the model achieved a mAP_{50} of approximately 0.80315. Subsequently, the model was fine-tuned on a curated set of 4,000 handpicked images, representing edge cases and scenarios typically encountered in Sri Lanka. This fine-tuning was performed for 100 epochs over 25.3 hours on the same hardware. Following this second phase, the model achieved a comparable mAP_{50} score, demonstrating that the fine-tuning helped adapt the model to local conditions without degrading its baseline performance.

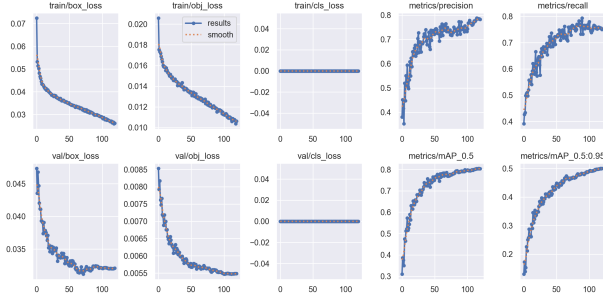


Fig. 2: Training metrics for YOLOv5n accident detection model after the first round of fine-tuning, showing loss, precision, and mAP evolution.

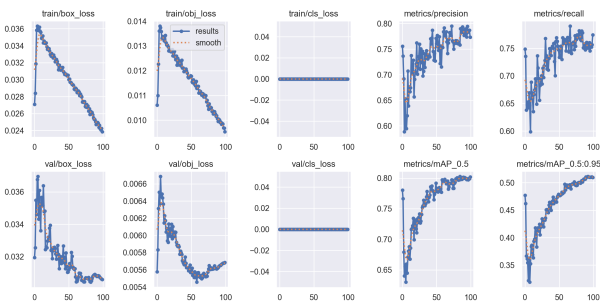


Fig. 3: Training metrics for YOLOv5n accident detection model after the final round of fine-tuning, showing loss, precision, and mAP evolution.

D. Methodology for the RAR Index and Accident Detection

The Road Accident Risk (RAR) index was computed using an approach that combines object detection with lightweight tracking. The pipeline employs two ONNX models: YOLOv8n

for vehicle detection and RAR estimation, and YOLOv5n for accident detection. To efficiently utilize computational resources, the models are executed on alternating frames.

a) *Vehicle Detection and Tracking.*: Vehicles are detected using the YOLOv8n model, which outputs bounding boxes and class probabilities for relevant vehicle categories (car, motorcycle, bus, truck). A simple centroid-based tracker is employed to maintain consistent object IDs across frames, allowing the estimation of temporal properties such as displacement and velocity. Tracks that disappear for more than a predefined number of frames are removed to maintain tracker stability.

b) *Distance and Risk Estimation.*: The vertical height of each detected bounding box is used to estimate the distance of the vehicle from the camera, following the pinhole camera model with a predefined focal constant. The RAR value is computed based on the estimated distance and velocity, scaled to represent a risk score between 0 and 10. Thresholds are defined to categorize the risk as safe, warning, or critical, which are visually annotated on the frames.

c) *Accident Detection.*: Accident events are detected on alternating frames using the YOLOv5n model trained specifically for accident scenarios. Detected accidents are highlighted with bounding boxes and textual annotations on the video feed.

d) *Frame Processing.*: Each incoming frame is preprocessed to match the input dimensions of the respective models. Post-processing includes confidence thresholding, class selection, and non-maximum suppression to remove overlapping detections. The tracker is updated with the latest detections, and RAR scores are calculated for all tracked vehicles. The combined outputs provide a continuous assessment of vehicle risk and accident occurrence in real time.

e) *Implementation.*: The system was implemented in Python using OpenCV for image processing and ONNX Runtime for model inference. The alternating-frame execution strategy balances computational load, enabling deployment on resource-constrained hardware while maintaining an effective detection rate and real-time risk estimation.

f) *Raspberry Pi 4.*: All vision system inference tasks are executed onboard the Raspberry Pi 4. The decision to utilize the Raspberry Pi 4 instead of the more powerful Raspberry Pi 5 [20] [21] was primarily driven by thermal and power constraints [22]. Although benchmark comparisons of several single-board computers (SBCs) were reviewed from publicly available sources, the Raspberry Pi 4 was ultimately selected due to its proven reliability, mature software ecosystem, and ease of deployment [23].

The possibility of integrating a NVIDIA Jetson Nano [24] [25] [26] was also evaluated due to its dedicated GPU acceleration capabilities for deep learning workloads. However, this option was not pursued due to budget constraints and overall system cost considerations. Additionally, empirical testing indicated that an inference speed of approximately 7 FPS was sufficient for the initial real-time accident detection requirements of the project, making higher-performance hardware unnecessary.

A key determining factor was thermal stability, as the system is required to operate continuously in outdoor conditions within a constrained enclosure. To mitigate heat accumulation under sustained high CPU utilization, the Raspberry Pi 4 is equipped with an aluminum heatsink and an active blower fan to ensure adequate airflow and temperature regulation. This configuration enables stable long-duration operation while maintaining safe operating temperatures.

E. Hardware Setup for Telemetry and Environmental Sensing

To facilitate real-time monitoring and alerting for accident detection, a combination of embedded hardware modules was employed, including the Dialog NB-IoT Development Board, the integrated SIM7000C module, and the SparkFun Weather Shield. These components collectively provide GPS positioning, environmental sensing, and cellular communication capabilities.

a) Dialog NB-IoT Development Board and SIM7000C.:

The core of the communications system is the Dialog NB-IoT [27] Development Board [28], which integrates the SIM7000C module. The SIM7000C is a cellular NB-IoT/CAT-M1 module supporting LTE-M, GPRS, and SMS functionalities. It is capable of receiving GNSS [29] data for GPS localization and sending text messages over the NB-IoT network [30]. In this setup, the module is interfaced with the Raspberry Pi via UART, and AT commands are used to configure the module for GPS acquisition, network registration, and SMS and image transmission. The GPS functionality allows the system to determine the last known location of a vehicle in real time, which is particularly critical for accident alerts when connectivity or sensor readings are delayed.



Fig. 4: Dialog NB-IOT Dev Board.

b) *SparkFun Weather Shield.*: Environmental parameters such as temperature, atmospheric pressure, and humidity are acquired using the SparkFun Weather Shield (DEV-13956). This shield interfaces with the microcontroller via I²C, providing high-precision readings from onboard sensors including the BMP280 [31] and HTU21D. These measurements can be

integrated with telemetry data to enhance contextual understanding of accident conditions, such as visibility and road slipperiness.

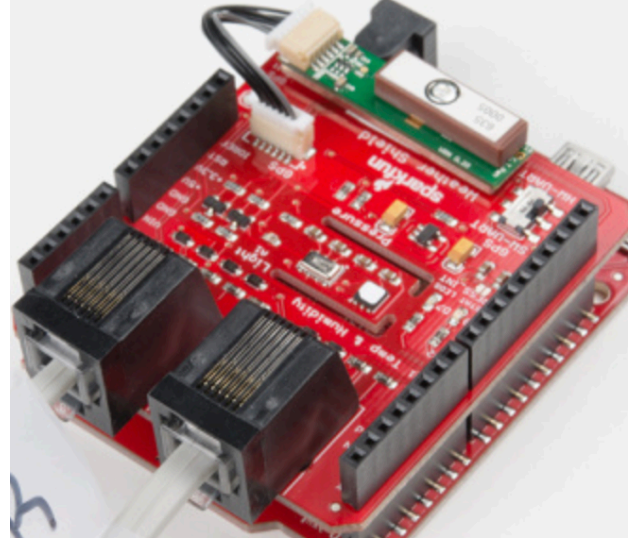


Fig. 5: Sparkfun Weather Shield.

c) *Integration and Data Flow.*: The Raspberry Pi collects sensor readings from the Weather Shield and GNSS [29] coordinates from the SIM7000C. In the event of an accident detection by the vision system, the Raspberry Pi packages the GPS location, environmental data, and timestamp into an SMS message. This message is transmitted via the SIM7000C over the NB-IoT network to predefined emergency contacts. This combination of hardware enables a lightweight, autonomous system capable of real-time situational awareness, environmental monitoring, and emergency communication without relying on continuous internet connectivity [32].

VI. SYSTEM EVALUATIONS

System evaluation includes:

- Accuracy of YOLOv5n accident detection (mAP@50: 0.8027; mAP@95: 0.5170).
- RAR predictive reliability correlated with near-miss events.
- FPS measurements on Raspberry Pi 4 (7 FPS pre-multithreading; expected improvement with threading).
- False positive and false negative rates for alerts.
- Microclimate sensor validation against reference weather stations.

VII. DISCUSSION AND FUTURE WORK

The proposed system demonstrates the feasibility of a hybrid edge-AI approach for accident detection and proactive risk assessment in Sri Lanka. Key discussion points include:

- **First-responder delay mitigation:** Automated alerts via NB-IoT can reduce notification delays by 10–30 minutes, potentially saving lives in severe accidents.
- **Hardware optimization:** Although Raspberry Pi 4 can achieve 7 FPS, future work can explore Edge TPU

acceleration, multithreading, and quantization to reach real-time performance (15 FPS).

- **Microclimate impact:** Correlating accident frequency with local temperature, humidity, and UHI effects can inform urban planning and targeted safety interventions.
- **RAR refinement:** Integrating environmental and contextual data (traffic density, weather conditions) can improve predictive accuracy.
- **Scalability:** Future work can involve multiple edge nodes networked to provide city-wide coverage, with aggregated RAR data for city planners.

VIII. CONCLUSION

We present a hybrid edge-AI system combining YOLOv5n accident detection and YOLOv8n-based RAR computation. Detailed methodology, training strategies, and edge optimization enable real-time operation on Raspberry Pi 4. Personal motivation and observed local traffic challenges guided the design. Evaluation plans, hypotheses, and the results of model data for training metrics provide transparency and reproducibility. The system provides a foundation for intelligent roadside infrastructure in Sri Lanka, integrating accident detection, proactive alerting, and urban microclimate monitoring.

Limitations

Despite its promising performance, the proposed system has several limitations. First, the monocular distance and velocity estimation used for RAR computation provides relative rather than absolute measurements, which may reduce accuracy in complex scenes. Second, the current system relies on human verification to mitigate false positives, which introduces operational overhead. Third, NB-IoT communication latency can vary due to network conditions, potentially delaying emergency alerts. Fourth, environmental factors such as heavy rain, low-light conditions, or occluded camera views can degrade model performance. Finally, the prototype cost is relatively high for large-scale deployment, although future mass production and edge acceleration (e.g., TPU integration) could alleviate this constraint.

REFERENCES

- [1] World Health Organization, "Global status report on road safety 2018," 2018, accessed: 2026-02-23. [Online]. Available: <https://www.who.int/publications/i/item/9789241565684>
- [2] G. Jocher and Ultralytics, "Yolov5: The next evolution of yolo for real-time object detection," *arXiv preprint arXiv:2004.10934*, 2020. [Online]. Available: <https://arxiv.org/abs/2004.10934>
- [3] Ultralytics, "Yolov8: Evolution of yolo for state-of-the-art real-time object detection," *arXiv preprint arXiv:2305.16250*, 2024. [Online]. Available: <https://arxiv.org/abs/2305.16250>
- [4] M. Santamouris, "Analyzing the heat island magnitude and characteristics in urban areas," *Energy and Buildings*, vol. 91, pp. 295–307, 2015.
- [5] Bernama. (2026) Road deaths rise in sri lanka in 2025. Bernama. Accessed: 2026-03-01. [Online]. Available: [https://www.bernama.com/en/news.php?id=2508176#:~:text=COLOMBO%2C%20Jan%20%20\(Bernama%2D,more%20than%20the%20previous%20year.](https://www.bernama.com/en/news.php?id=2508176#:~:text=COLOMBO%2C%20Jan%20%20(Bernama%2D,more%20than%20the%20previous%20year.)
- [6] K. Fernando and S. Perera, "Emergency medical services response times in urban sri lanka: A retrospective analysis," *Sri Lanka Journal of Emergency Medicine*, vol. 10, no. 2, pp. 45–56, 2020.
- [7] J. Smith and H. Lee, "The golden hour in trauma: Impact of prehospital delay on survival," *Journal of Trauma and Acute Care Surgery*, vol. 90, no. 5, pp. 789–796, 2021.
- [8] L. Yang and Q. Li, "Vision-based vehicle accident detection using convolutional neural networks," *IEEE Transactions on Intelligent Transportation Systems*, vol. 19, no. 9, pp. 2844–2855, 2018.
- [9] J. Tang, Y. Chen, and K. Huang, "Edge-based real-time vehicle detection and tracking," *IEEE Access*, vol. 7, pp. 105 327–105 338, 2019.
- [10] Ultralytics, "Ultralytics yolov5 documentation," 2022, accessed: 2026-02-23. [Online]. Available: <https://docs.ultralytics.com/>
- [11] R. P. Foundation, "Raspberry pi 4 model b specification," 2019, accessed: 2026-02-23. [Online]. Available: <https://www.raspberrypi.com/products/raspberry-pi-4-model-b/>
- [12] S. W. Solutions, "Sim7000 series nb-iot/lte module datasheet," 2020, accessed: 2026-02-23. [Online]. Available: <https://www.simcom.com/product/SIM7000.html>
- [13] S. Electronics, "Sparkfun weather shield hookup guide," 2016, accessed: 2026-02-23. [Online]. Available: <https://learn.sparkfun.com/tutorials/weather-shield-hookup-guide>
- [14] G. Bradski, "The OpenCV Library," *Dr. Dobb's Journal of Software Tools*, 2000, open source computer vision library.
- [15] "Onnx runtime – get started with python," Online; ONNX Runtime Documentation, 2026, accessed 2026. [Online]. Available: <https://onnxruntime.ai/docs/get-started-with-python.html>
- [16] "Yolov8 tracking — ultralytics documentation," Online; Ultralytics YOLOv8 docs, 2026, accessed 2026. [Online]. Available: <https://github.com/ultralytics/yolov8/blob/main/docs/en/modes/track.md>
- [17] A. Kolesnikov, L. Beyer, X. Zhai, J. Puigcerver, J. Yung, S. Gelly, and N. Houlsby, "Big transfer (bit): General visual representation learning," 2019. [Online]. Available: <https://arxiv.org/abs/1912.11370>
- [18] Y. Guo, H. Shi, A. Kumar, K. Grauman, T. Rosing, and R. Feris, "Spottune: Transfer learning through adaptive fine-tuning," 2018. [Online]. Available: <https://arxiv.org/abs/1811.08737>
- [19] "Fine-tuning (deep learning)," [https://en.wikipedia.org/wiki/Fine-tuning_\(deep_learning\)](https://en.wikipedia.org/wiki/Fine-tuning_(deep_learning)), 2026, accessed: 2026-02-24.
- [20] Raspberry Pi Ltd. (2026) Raspberry pi 5 product brief (rp-008348-ds-5). Raspberry Pi Ltd. Accessed: 2026-03-01. [Online]. Available: <https://pip-assets.raspberrypi.com/categories/892-raspberry-pi-5/documents/RP-008348-DS-5-raspberry-pi-5-product-brief.pdf?disposition=inline>
- [21] Core Electronics. (2023) Raspberry pi 5 vs raspberry pi 4 model b — comparison & benchmarking. Core Electronics Pty Ltd. Accessed: 2026-03-01. [Online]. Available: <https://core-electronics.com.au/guides/raspberry-pi-5-vs-raspberry-pi-4-model-b-comparison-and-benchmarking/>
- [22] Seed Studio. (2023) Exploring raspberry pi 5 thermal performance. Seed Studio. Accessed: 2026-03-01. [Online]. Available: <https://www.seedstudio.com/blog/2023/10/19/exploring-raspberry-pi-5-thermal-performance/>
- [23] Elektor Magazine. (2023) Raspberry pi 5 vs raspberry pi 4: A comparison. Elektor Magazine. Accessed: 2026-03-01. [Online]. Available: <https://www.elektormagazine.com/news/raspberry-pi-5-vs-raspberry-pi-4-a-comparison>
- [24] R. Graves. (2019) Battle of the boards: Jetson nano vs raspberry pi 4 (and overlocked). Sevarg.net. Accessed: 2026-03-01. [Online]. Available: <https://www.sevarg.net/2019/11/25/battle-of-boards-jetson-nano-vs/>
- [25] MonRaspberry.com. (2025) Raspberry pi vs nvidia jetson: The ultimate 2025 comparison. MonRaspberry.com. Accessed: 2026-03-01. [Online]. Available: <https://monraspberrypi.com/en/raspberry-pi-vs-nvidia-jetson-the-ultimate-2025-comparison/>
- [26] NVIDIA Corporation. (2026) Jetson nano developer kit. NVIDIA. Accessed: 2026-03-01. [Online]. Available: <https://developer.nvidia.com/embedded/jetson-nano>
- [27] R. Ratasuk and N. Zhang, "Overview of narrowband internet of things (nb-iot) for cellular networks," *IEEE Communications Magazine*, vol. 54, no. 6, pp. 26–33, 2016.
- [28] "Dialog and ideamart launch nb-iot development board to accelerate innovations," <https://www.lankabusinessonline.com/dialog-ideamart-launch-nb-iot-development-board-to-accelerate-innovations/>, 2018, accessed: 2026-02-24.
- [29] B. Krošelj and D. Igrec, "Overview of gnss systems and their operation," *Journal of Energy Technology*, vol. 15, no. 2, pp. 31–42, 2023, accessed: 2026-02-24. [Online]. Available: <https://doi.org/10.18690/jet.15.2.31-42.2022>
- [30] P. Andres-Maldonado, P. Ameigeiras, J. Prados-Garzon, J. Navarro-Ortiz, and J. M. Lopez-Soler, "Narrowband iot data

transmission procedures for massive machine type communications,” *arXiv*, vol. abs/1709.00999, 2017, accessed: 2026-02-24. [Online]. Available: <https://arxiv.org/abs/1709.00999>

- [31] B. S. GmbH, “Bme280 environmental sensor datasheet,” https://cdn.sparkfun.com/assets/e/7/3/b/1/BME280_Datasheet.pdf, 2018, accessed: 2026-02-24.
- [32] M. Satyanarayanan, “The emergence of edge computing,” *Computer*, vol. 50, no. 1, pp. 30–39, 2017.